



Amazon CloudFront — Complete Notes

Content Delivery Network (CDN) · Cache & deliver content from the edge

A reader-friendly, all-in-one reference for Amazon CloudFront.

1. 🌐 What is CloudFront

Amazon CloudFront is AWS's **Content Delivery Network (CDN)**. It caches your content at a global network of **edge locations** so users are served from a nearby point of presence instead of reaching across the world to your origin — cutting **latency**, offloading your servers, and adding a **security** layer.

💡 **Mental model:** CloudFront is a **fleet of caches near your users**. The first request for an object fetches it from the **origin** and stores it at the edge; later requests are served from that cache — fast and cheap.

Common uses: static asset delivery (images/JS/CSS), accelerating dynamic APIs, video streaming, fronting an S3 static website, software downloads, and as a security/HTTPS layer in front of ALB/S3.

2. ⚙️ How It Works (Edge Caching)

```
User → nearest Edge Location —(cache HIT)—→ served instantly
      |
      └─(cache MISS)—→ Regional Edge Cache → Origin (S3 / ALB / server)
                                   |
                                   object cached at edge ← (for next time, per TTL)
```

1. A user request hits the **closest edge location** (200+ globally).
2. **Cache HIT** → the edge serves the cached object immediately.
3. **Cache MISS** → CloudFront fetches from the **origin** (often via a larger **Regional Edge Cache**), returns it, and **caches it at the edge** for the object's TTL.
4. Subsequent nearby requests are served from cache → low latency + less origin load.

3. 🧩 Core Concepts

Concept	Description
Distribution	The CloudFront config (its domain <code>dxxxx.cloudfront.net</code> + behaviors).
Edge Location	A global PoP that caches and serves content.
Regional Edge Cache	A larger mid-tier cache between edges and the origin.
Origin	Where the real content lives (S3, ALB, EC2, any HTTP server).
Cache Behavior	Rules (by path pattern) for how to cache/route requests.
Cache Key	What identifies a cached object (host, path, + chosen headers/query/cookies).
TTL	How long an object stays cached at the edge.
Invalidation	Force-remove cached objects before TTL expiry.

4. 🎯 Origins

CloudFront pulls content from an **origin**:

Origin	Use
S3 bucket	Static assets / static website hosting (lock down with OAC).
ALB / EC2 / custom HTTP	Dynamic apps and APIs.
MediaStore / MediaPackage	Video streaming.
Origin Group	Primary + secondary origin for origin failover (high availability).

- A single distribution can have **multiple origins** and route to them via cache behaviors (e.g. `/api/*` → ALB, everything else → S3).

5. 🔑 Cache Behaviors & Cache Key

- **Cache behaviors** match request **path patterns** (`/images/*`, `/api/*`) and set: target origin, allowed HTTP methods, **caching policy**, header/cookie/query forwarding, and whether to compress.
- The **cache key** decides whether two requests hit the same cached object. Keep it **minimal** — including unnecessary headers, cookies, or query strings fragments the cache and lowers the hit rate.
- **Cache policies** + **origin request policies** (managed or custom) control what's in the cache key vs what's merely forwarded to the origin.

💡 Higher **cache hit ratio** = lower latency + less origin cost. Forward only the fields that actually change the response.

6. 🕒 TTLs & Invalidation

- **TTL** governs cache lifetime at the edge — set via origin headers (`Cache-Control: max-age` , `Expires`) or CloudFront `min/default/max TTL` settings.
- **Cache busting** (versioned filenames like `app.4f3a.js`) is the cleanest way to ship updates — a new name is a new object, so caches never serve stale content.
- **Invalidation** forcibly removes objects from edge caches before TTL (e.g. `/index.html` or `/*`). First 1,000 paths/month free, then charged — so prefer **versioned filenames** over frequent invalidations.

7. 🛡️ Security (HTTPS, OAC, WAF)

- **HTTPS** everywhere — use **ACM certificates** (must be in **us-east-1** for CloudFront), enforce *Viewer Protocol Policy* = redirect-to-HTTPS, set min TLS version.
- **OAC (Origin Access Control)** — locks an **S3 origin** so it's reachable **only** through CloudFront, not directly (replaces the legacy OAI).
- **AWS WAF** — attach a web ACL to filter SQLi/XSS/bad bots and rate-limit at the edge.
- **AWS Shield** — DDoS protection (Standard is automatic; Advanced for more).
- **Field-level encryption** for sensitive form fields.

8. ⚠️ Restricting Access

Mechanism	Restricts by
Signed URLs	Time-limited access to one file (premium downloads).
Signed Cookies	Time-limited access to many files (whole sections/streams).
Geo Restriction	Allow/deny by country (compliance, licensing).
OAC + bucket policy	Force S3 access only via CloudFront.
WAF rules	IP allow/deny, rate limits, managed rule groups.

9. ⚡ Edge Compute (Lambda@Edge & Functions)

Run code at the edge to customize requests/responses without a round-trip to the origin:

	CloudFront Functions	Lambda@Edge
Runtime	Lightweight JS	Node.js / Python (full Lambda)
Latency / scale	Sub-ms, massive scale	Milliseconds
Triggers	Viewer request/response	All 4 (viewer + origin request/response)
Use	Header rewrites, redirects, URL normalization, simple auth	Heavier logic, calls to other AWS services, A/B, auth

Use **CloudFront Functions** for tiny, ultra-fast viewer-side tweaks; **Lambda@Edge** when you need real compute or origin-side logic.

10. ⚖️ CloudFront vs S3 vs Global Accelerator

	CloudFront	S3 (alone)	Global Accelerator
Purpose	Cache + deliver content at the edge	Store objects	Route TCP/UDP to nearest healthy endpoint
Caches content?	✅ Yes	❌ No	❌ No
Layer	HTTP(S) content (L7)	Storage	Network (L4), static anycast IPs
Use with	Any origin (often S3/ALB)	Origin for CloudFront	Non-HTTP / fast failover, gaming, APIs

Front an S3 static site with **CloudFront** for caching, HTTPS, custom domain, and OAC. Use **Global Accelerator** (not CloudFront) for non-HTTP protocols or static-IP fast failover.

11. 💰 Pricing

You pay for:

1. **Data transfer out** to the internet (varies by region/edge), tiered by volume.
2. **HTTP/HTTPS requests** (per 10,000).
3. **Invalidations** beyond the free 1,000 paths/month.
4. **Edge compute** (CloudFront Functions invocations / Lambda@Edge) and optional features (field-level encryption, real-time logs).

🔑 **Cost tips:** maximize **cache hit ratio** (minimal cache key, good TTLs) to cut origin transfer · transfer from S3/EC2 **into** CloudFront is free · use **versioned filenames** instead of paid invalidations · pick a **price class** (limit to certain regions) if global edges aren't needed.

12. 📖 Common CLI Commands

```
aws cloudfront create-distribution --distribution-config file:///dist.json
aws cloudfront list-distributions
aws cloudfront get-distribution --id E123ABC

# Invalidate cached objects (prefer versioned filenames over frequent invalidations)
aws cloudfront create-invalidation --distribution-id E123ABC --paths "/index.html" "/css/*"

aws cloudfront update-distribution --id E123ABC --distribution-config file:///dist.json --if-match <ETag>
```

13. ✅ Best Practices

- **Maximize cache hit ratio** — minimal cache key; forward only headers/cookies/query that change the response.
 - Use **versioned filenames** (`app.<hash>.js`) for instant, free cache busting instead of invalidations.
 - **Lock S3 origins with OAC** so content is only reachable via CloudFront.
 - **Enforce HTTPS** (ACM cert in us-east-1, redirect-to-HTTPS, modern TLS).
 - Add **WAF + Shield** for security; **geo restriction** for compliance/licensing.
 - Use **Origin Groups** for origin failover (HA); set sensible **TTLs** per content type.
 - Use **CloudFront Functions** for light edge logic, **Lambda@Edge** for heavier needs.
 - Enable **compression** (gzip/Brotli) and pick a **price class** to control cost.
-

14. 🧠 Quick Mental Model

- **CloudFront = global CDN**: caches content at 200+ **edge locations** near users → low latency + origin offload.
 - First request = **MISS** (fetch from **origin**, cache it); later = **HIT** (served from edge per **TTL**).
 - A **distribution** routes by **cache behaviors** (path patterns) to **origins** (S3, ALB, custom).
 - The **cache key** decides reuse — keep it minimal for a high **hit ratio**.
 - Update content via **versioned filenames** (free) over **invalidations** (paid beyond 1,000/mo).
 - Secure with **HTTPS (ACM in us-east-1)**, **OAC** (lock S3), **WAF/Shield**, signed URLs/cookies, geo restriction.
 - Customize at the edge with **CloudFront Functions** (tiny/fast) or **Lambda@Edge** (full compute).
-

15. ? Common Interview Questions

Rapid-fire questions interviewers ask about CloudFront:

- **Q: What is CloudFront and why use it?** — A CDN that caches content at global edge locations, reducing latency, offloading the origin, and adding HTTPS/security.
- **Q: Cache HIT vs MISS?** — HIT = served from the edge cache. MISS = edge fetches from the origin, then caches it for the TTL.
- **Q: How do you serve updated content immediately?** — Use **versioned filenames** (preferred) or create an **invalidation** to purge cached paths.

- **Q: How do you secure an S3 origin?** — **Origin Access Control (OAC)** + a bucket policy so S3 is reachable only through CloudFront, not directly.
- **Q: Where must the ACM cert live for CloudFront?** — In **us-east-1** (N. Virginia), regardless of where your origin is.
- **Q: Signed URL vs signed cookie?** — Signed URL = time-limited access to one file; signed cookie = access to many files without changing URLs.
- **Q: CloudFront Functions vs Lambda@Edge?** — Functions = lightweight JS, sub-ms, viewer-side (header rewrites/redirects). Lambda@Edge = full runtime, origin-side too, heavier logic.
- **Q: CloudFront vs Global Accelerator?** — CloudFront caches HTTP(S) content at the edge. Global Accelerator doesn't cache — it routes TCP/UDP over the AWS backbone to the nearest healthy endpoint with static anycast IPs.
- **Q: How do you improve cache hit ratio?** — Minimize the cache key (forward only necessary headers/cookies/query strings) and set good TTLs.
- **Q: How do you restrict content by country?** — **Geo restriction** (allow/deny lists) on the distribution.

 Notes compiled as a quick reference for Amazon CloudFront.